

Task 1: Write a C program to demonstrate the memory layout of a process by displaying the addresses of code, global variables, static variables, heap variables, and stack variables. Allocate memory dynamically using malloc() and display the corresponding addresses.

Code:

```
int main()
{
    int stack_var = 30;
    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
    {
        perror("malloc");
        return 1;
    }

    *heap_var = 40;

    printf("==== Linux Process Memory Layout ==== \n\n");

    printf("PID           : %d\n", getpid());

    printf("Code/Text address : %p\n", (void *)my_function);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable    : %p\n", (void *)&static_var);
    printf("Heap variable      : %p\n", (void *)heap_var);
    printf("Stack variable     : %p\n", (void *)&stack_var);

    printf("\n===== \n");

    printf("\nAddress interpretation:\n");
    printf("Code/Text  -> my_function()\n");
    printf("Global     -> global_var\n");
    printf("Static     -> static_var\n");
    printf("Heap       -> memory allocated by malloc()\n");
    printf("Stack      -> local variable in main()\n");

    free(heap_var);

    return 0;
}
```

Output:

```
karthik@Marthikeya:~$ mkdir -p ~/memory_layout
cd ~/memory_layout
karthik@Marthikeya:~/memory_layout$ nano memory.c
karthik@Marthikeya:~/memory_layout$ gcc memory.c -o memory
memory.c: In function 'main':
memory.c:29:44: warning: implicit declaration of function 'getpid' [-Wimplicit-function-declaration]
   29 |         printf("PID           : %d\n", getpid());
      |                                ^
karthik@Marthikeya:~/memory_layout$ ./memory
==== Linux Process Memory Layout ====

PID           : 2234

Code/Text address : 0x610e809f5209
Global variable   : 0x610e809f8010
Static variable    : 0x610e809f8014
Heap variable      : 0x610e998f22a0
Stack variable     : 0x7ffccf6af1fc

=====

Address interpretation:
Code/Text  -> my_function()
Global     -> global_var
Static     -> static_var
Heap       -> memory allocated by malloc()
Stack      -> local variable in main()
karthik@Marthikeya:~/memory_layout$ |
```

Task 2: Write a C program to dynamically allocate memory using malloc(), display the process ID and heap address, and inspect the process memory mappings using /proc/<PID>/maps to identify the memory regions.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int *heap = malloc(1024 * 1024);

    if (heap == NULL) {
        return 1;
    }

    printf("PID: %d\n", getpid());
    printf("Heap address: %p\n", (void *)heap);
    printf("Press Enter to exit...\n");

    getchar();

    free(heap);
    return 0;
}
```

Output:

```
karthik@Karthikeya:~/memory_layout$ mkdir -p ~/memory_analysis
cd ~/memory_analysis
karthik@Karthikeya:~/memory_analysis$ nano memory_analysis.c
karthik@Karthikeya:~/memory_analysis$ gcc memory_analysis.c -o memory_analysis
karthik@Karthikeya:~/memory_analysis$ ./memory_analysis
PID: 2272
Heap address: 0x777f16eff010
Press Enter to exit...
```

```
karthik@Karthikeya:~/memory_analysis$ |

karthik@Karthikeya:~$ cat /proc/2272/maps
5619af08c000-5619af08d000 r--p 00000000 08:30 18355 /home/karthik/memory_analysis/memory_analysis
5619af08d000-5619af08e000 r-xp 00001000 08:30 18355 /home/karthik/memory_analysis/memory_analysis
5619af08e000-5619af08f000 r--p 00002000 08:30 18355 /home/karthik/memory_analysis/memory_analysis
5619af08f000-5619af090000 r--p 00002000 08:30 18355 /home/karthik/memory_analysis/memory_analysis
5619af090000-5619af091000 rw-p 00003000 08:30 18355 /home/karthik/memory_analysis/memory_analysis
5619d8635000-5619d8656000 rw-p 00000000 00:00 0 [heap]
777f16eff000-777f17000000 rw-p 00000000 00:00 0
777f17000000-777f17020000 r--p 00000000 08:30 49357 /usr/lib/x86_64-linux-gnu/libc.so.6
777f17020000-777f171b1000 r-xp 00028000 08:30 49357 /usr/lib/x86_64-linux-gnu/libc.so.6
777f171b1000-777f17200000 r--p 001b1000 08:30 49357 /usr/lib/x86_64-linux-gnu/libc.so.6
777f17200000-777f17204000 r--p 001ff000 08:30 49357 /usr/lib/x86_64-linux-gnu/libc.so.6
777f17204000-777f17206000 rw-p 00203000 08:30 49357 /usr/lib/x86_64-linux-gnu/libc.so.6
777f17206000-777f17213000 rw-p 00000000 00:00 0
777f1724a000-777f1724d000 rw-p 00000000 00:00 0
777f17253000-777f17255000 rw-p 00000000 00:00 0
777f17255000-777f17259000 r--p 00000000 00:00 0 [vvar]
777f17259000-777f1725b000 r--p 00000000 00:00 0 [vvar_vclock]
777f1725b000-777f1725d000 r-xp 00000000 00:00 0 [vdso]
777f1725d000-777f1725e000 r--p 00000000 08:30 48264 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
777f1725e000-777f17289000 r-xp 00001000 08:30 48264 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
777f17289000-777f17293000 r--p 0002c000 08:30 48264 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
777f17293000-777f17295000 r--p 00036000 08:30 48264 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
777f17295000-777f17297000 rw-p 00038000 08:30 48264 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffdef036000-7ffdef057000 rw-p 00000000 00:00 0 [stack]
```